React kézikönyv: debug, alapok, Context, adatbázis

Oct 4, 2026 · @Györgyövics istván

1. Debug lépésről lépésre

Minden hibánál ugyanaz a sorrend: reprodukálod, megnézed a Network fület, abból eldöntöd, frontend vagy backend, majd szűkíted, javítod, újratesztelsz.

1. **Reprodukáld.** Írd le egy mondatban: mit csinálsz, mi történik, mit vártál volna.
2. **F12 → Console fül.** Van piros hiba? Kattints a fájlnévre a hiba mellett: odaugrik a hibás sorra.
3. **F12 → Network fül, frissíts.** Szűrj a Fetch/XHR-re. Kattints a kérésre: **Headers** (URL, metódus, státuszkód), **Payload** (mit küldtél), **Response** (mit kaptál, pontos mezőnevek).
4. **Döntsd el a státuszkódból, hol a hiba** (lenti táblázat).
5. **React DevTools → Components fül.** Kattints a komponensre: látod a props-ot, a state-et és a context értékét. Változik, amikor kellene?
6. **Szűkítsd `console.log`-gal.** Írasd ki a gyanús változót a kritikus ponton: `console.log('mentes elott', form);`
7. **Javíts egyet, tesztelj újra.** Mindig egy hibát egyszerre, mert egy hiba elfedhet egy másikat.

Státusz	Jelentés	Hol keresd
nincs kérés	a kód el sem küldte	React: lefut-e a függvény, a useEffect, az onSubmit
200 / 201	a backend rendben	React: feldolgozás, mezőnevek, state frissítés
400 / 422	hibás adat ment	Payload: mezőnév, szám vs. szöveg; Response: melyik mező
401 / 403	nincs bejelentkezve / nincs jog	token, Auth context, fejléc
404	rossz URL	elírás az `api.js`-ben, hiányzó végpont
419	CSRF token hiányzik (Laravel)	kérés fejléce

Státusz	Jelentés	Hol keresd
500	a szerver kódja hibás	backend log: `storage/logs/laravel.log`
CORS hiba	más domainről tiltott	backend beállítás vagy proxy

Tipikus React-hibák, ezeket nézd meg először

- [ ] `useEffect` függőségi tömbje hiányzik (végtelen kérés) vagy hiányos (nem tölt újra, amikor kellene)
- [ ] state közvetlen módosítása: `push`, `obj.x = …` → nem rajzol újra
- [ ] rossz setter vagy elírt mezőnév (`p.title` vs. `p.name`)
- [ ] hiányzó `e.preventDefault()` → újratölt az oldal
- [ ] szöveg megy szám helyett (`"5"` vs. `5`) → `Number()`
- [ ] Context: hiányzó Provider, vagy a value-ból kimaradt egy függvény
- [ ] listánál hiányzó vagy nem egyedi `key`
- [ ] `loading` sosem lesz `false` (csak a catch-ben állítod)
- [ ] `fetch` 404-nél nem dob hibát: `res.ok` ellenőrzés kell

Gyakori hibaüzenetek fordítása

Üzenet	Mit jelent
`Cannot read properties of undefined (reading 'name')`	az objektum még nincs betöltve vagy rossz a mezőnév → `user?.name`
`Cannot destructure property 'x' of ... as it is null`	`useContext` null-t adott: hiányzik a Provider
`x is not a function`	elírt név, vagy nincs benne a context value-ban / props-ban
`Assignment to constant variable`	`const`-nak adtál értéket, settert kell hívni
`Each child in a list should have a unique "key" prop`	hiányzik a `key` a `map`-nél
`Too many re-renders`	renderelés közben hívsz settert, pl. `onClick={setDb(1)}` helyett `onClick={() => setDb(1)}`
`Unexpected token '<' ... is not valid JSON`	HTML jött JSON helyett: rossz URL vagy szerverhiba oldal

2. React alapok

A React-ben minden komponens: egy függvény, ami JSX-et ad vissza, és újrarajzolódik, ha a props-a, a state-je vagy a használt context változik.

Komponens és props

```
// a név NAGYBETŰVEL kezdődik, a props egy objektum
function Kartya({ cim, ar }) {
  return <div className="card p-3">{cim}: {ar} Ft</div>;
}

// használat: a szülő adja át az adatot
<Kartya cim="Egér" ar={4990} />
```

JSX szabályok: `className` (nem class), `htmlFor` (nem for), `{}` közé JS kifejezés, minden tag lezárva (`<img />`), események camelCase-ben (`onClick`, `onChange`, `onSubmit`).

useState – a komponens saját adata

```
const [db, setDb] = useState(0);            // [érték, módosító], kezdőérték

setDb(db + 1);                              // egyszerű érték
setDb(elozo => elozo + 1);                  // ha az előző értékből számolsz:
biztosabb

// TÖMB és OBJEKTUM: mindig ÚJ példányt adj!
setLista([...lista, ujElem]);               // hozzáadás
setLista(lista.filter(x => x.id !== id));   // törlés
setLista(lista.map(x => x.id === id ? { ...x, db: x.db + 1 } : x));  //
módosítás
setForm({ ...form, nev: 'Pista' });         // objektum egy mezője
```

Soha ne módosítsd közvetlenül (`lista.push(...)`, `form.nev = ...`): a React azt látja, hogy ugyanaz a tömb, és nem rajzol újra.

useEffect – mellékhatások (adatletöltés, időzítő, localStorage)

```
useEffect(() => {
  // ide jön, aminek a megjelenés UTÁN kell lefutnia
}, [fuggosegek]);
```

Függőségi tömb	Mikor fut le	Mire
`[]`	egyszer, az első megjelenéskor	kezdő adatbetöltés
`[user]`	elsőre, és amikor a `user` változik	betöltés bejelentkezés után, szűrés
nincs tömb	minden rajzolás után	szinte soha, végtelen ciklus veszély

Szabály: ami a useEffect-en belül használt state vagy prop, és változhat, annak a tömbben a helye.

```
// a mintá, amit minden feladatban használsz
const [adatok, setAdatok] = useState([]);
const [loading, setLoading] = useState(true);
const [hiba, setHiba] = useState(null);

useEffect(() => {
  apiGet('/suppliers')
    .then(setAdatok)
    .catch(e => setHiba(e.message))
    .finally(() => setLoading(false));      // sikernél és hibánál is!
}, []);

// takarítás (cleanup): időzítő, feliratkozás leállítása
useEffect(() => {
  const t = setTimeout(() => setUzenet(null), 3000);
  return () => clearTimeout(t);              // ez fut le a következő
változás előtt
}, [uzenet]);
```

Események és űrlap (kontrollált input)

```
const [form, setForm] = useState({ name: '', email: '' });

// egy kezelő minden mezőhöz: az input name attribútuma dönti el, melyik mező
const handleChange = (e) => setForm({ ...form, [e.target.name]:
e.target.value });

async function handleSubmit(e) {
  e.preventDefault();                        // ne töltse újra az oldalt
  // ... mentés
}

<form onSubmit={handleSubmit}>
  <input name="name" value={form.name} onChange={handleChange} />
  <button>Mentés</button>
</form>
```

Lista és feltételes megjelenítés

```
{loading && <p>Betöltés…</p>}
{hiba && <div className="alert alert-danger">{hiba}</div>}
{user ? <Profil /> : <Login />}

<ul>
  {lista.map(t => <li key={t.id}>{t.name}</li>)}   {/* key kötelező */}
</ul>
```

További hookok röviden

Hook	Mire	Példa
`useContext`	közös adat elérése props nélkül	lásd 3. fejezet
`useRef`	DOM elem elérése, újrarajzolást nem okozó érték	`inputRef.current.focus()`
`useMemo`	drága számítás eredményének megjegyzése	`useMemo(() => lista.filter(...), [lista, szuro])`
`useCallback`	függvény megjegyzése (props-ként átadott függvénynél)	`useCallback(() => {...}, [deps])`

Hook-szabályok: csak a komponens legfelső szintjén hívható (nem `if`-ben, nem ciklusban), és csak React komponensben vagy saját `use...` hookban.

3. Context részletesen

A Context egy „közös doboz”: amit a Provider-be teszel, azt bármelyik komponens eléri alatta, props-ok továbbadogatása nélkül. Akkor használd, ha sok komponensnek kell ugyanaz az adat: bejelentkezett felhasználó, téma, kosár, értesítések, közös lista.

A 3 lépés, egy fájlban

```
// src/context/CartContext.jsx
import { createContext, useContext, useState } from 'react';

// 1) LÉTREHOZÁS
const CartContext = createContext(null);

// 2) PROVIDER: itt van a state és a függvények, amiket megosztasz
export function CartProvider({ children }) {
  const [cart, setCart] = useState([]);

  function addToCart(product) {
    setCart(elozo => [...elozo, product]);      // új tömb!
  }
  function removeFromCart(id) {
    setCart(elozo => elozo.filter(t => t.id !== id));
  }

  // a value-ba MINDEN bekerül, amit a komponensek használni akarnak
  return (
    <CartContext.Provider value={{ cart, addToCart, removeFromCart }}>
      {children}
    </CartContext.Provider>
  );
}

// 3) SAJÁT HOOK a használathoz (+ érthető hiba, ha nincs Provider)
export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart csak CartProvider-en belül
használható!');
  return ctx;
}
```

Becsomagolás és használat

```
// main.jsx vagy App.jsx – a Provider KÍVÜL legyen mindenen, ami használja
<AuthProvider>
  <CartProvider>
    <App />
  </CartProvider>
</AuthProvider>

// bármelyik komponensben, bármilyen mélyen
function KosarIkon() {
  const { cart } = useCart();
  return <span>Kosár: {cart.length}</span>;
}
function TermekKartya({ product }) {
  const { addToCart } = useCart();
  return <button onClick={() => addToCart(product)}>Kosárba</button>;
}
```

Context + API: a leggyakoribb munkahelyi minta

```
export function SuppliersProvider({ children }) {
  const [suppliers, setSuppliers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    apiGet('/suppliers')
      .then(setSuppliers)
      .catch(e => setError(e.message))
      .finally(() => setLoading(false));
  }, []);

  async function addSupplier(data) {
    const saved = await apiPost('/suppliers', data);   // a backend adja
vissza az id-t
    setSuppliers(elozo => [...elozo, saved]);
    return saved;
  }

  return (
    <SuppliersContext.Provider value={{ suppliers, loading, error,
addSupplier }}>
```

```
      {children}
    </SuppliersContext.Provider>
  );
}
```

Gyakori Context-hibák

Tünet	Ok	Javítás
`Cannot destructure property ... as it is null`	a komponens nincs a Provider alatt	csomagold be a Providerrel
`addToCart is not a function`	kimaradt a `value`-ból, vagy más néven van	nézd meg a `value={{ ... }}` tartalmát
a szám nem frissül kattintásra	a state-et közvetlenül módosítod	új tömb/objektum a setterbe
két komponens különböző adatot lát	két külön Provider példány van	egy Provider legyen, magasabban
`Assignment to constant variable`	`theme = 'dark'` setter helyett	`setTheme('dark')`

Debugnál a React DevTools Components fülén kattints a Provider-re: a `value`-ban látod, mi van éppen a context-ben.

4. Recept: lista kiírása és hozzáadás – mit hova írj

A sorrend mindig: megnézed az API-t → `api.js` → context (vagy state) → lista komponens → űrlap komponens → beillesztés. Minden lépés után tesztelsz.

Lépés	Fájl	Mit írsz bele
1. API felderítése	böngésző, Network fül	URL, metódus, a válasz mezőnevei
2. API hívás	`src/api.js`	`getX`, `createX` függvény (vagy a közös `apiGet` / `apiPost`)
3. Közös adat	`src/context/XContext.jsx`	state, `useEffect` betöltés, `addX`, saját hook
4. Lista	`src/components/XList.jsx`	`useX()`, loading/hiba, `map` + `key`
5. Űrlap	`src/components/XForm.jsx`	kontrollált inputok, `handleSubmit`, `addX`

Lépés	Fájl	Mit írsz bele
6. Beillesztés	`App.jsx` / `main.jsx`	Provider köré, komponensek a helyükre

Hogyan deríted ki egy ismeretlen projektben, hol mi van

1. **Network fül**: milyen URL-eket hív az oldal, milyen mezőkkel jön az adat.
2. **VS Code,** **`Ctrl+Shift+F`** : keress rá a `fetch` , `axios` , `createContext` , `Provider` szavakra, vagy az URL egy darabjára.
3. **Mappák**: `src/api` vagy `src/services` (hívások), `src/context` (közös adat), `src/components` (UI), `src/pages` (oldalak), `src/hooks` (saját hookok).
4. **`.env`** : `VITE_API_URL` vagy `REACT_APP_API_URL` – a backend címe.
5. **Másold a mintát**: keresd meg a leghasonlóbb meglévő funkciót, és ugyanazzal a szerkezettel építsd az újat.

Sablonkód – lista komponens

```
// src/components/SupplierList.jsx
import { useSuppliers } from '../context/SuppliersContext.jsx';

export default function SupplierList() {
  const { suppliers, loading, error } = useSuppliers();

  if (loading) return <p>Betöltés…</p>;
  if (error) return <div className="alert alert-danger">{error}</div>;
  if (suppliers.length === 0) return <p>Nincs adat.</p>;

  return (
    <table className="table">
      <thead><tr><th>Név</th><th>E-mail</th></tr></thead>
      <tbody>
        {suppliers.map(s => (
          <tr key={s.id}><td>{s.name}</td><td>{s.email}</td></tr>
        ))}
      </tbody>
    </table>
  );
}
```

Sablonkód – űrlap komponens

```
// src/components/SupplierForm.jsx
import { useState } from 'react';
import { useSuppliers } from '../context/SuppliersContext.jsx';

const URES = { name: '', email: '' };

export default function SupplierForm() {
  const { addSupplier } = useSuppliers();
  const [form, setForm] = useState(URES);
  const [hiba, setHiba] = useState(null);

  const handleChange = (e) => setForm({ ...form, [e.target.name]:
e.target.value });

  async function handleSubmit(e) {
    e.preventDefault();
    setHiba(null);
    try {
      await addSupplier(form);     // számnál: { ...form, price:
Number(form.price) }
      setForm(URES);
    } catch (err) {
      setHiba(err.message);
    }
  }

  return (
    <form onSubmit={handleSubmit} className="row g-2 mb-3">
      {hiba && <div className="alert alert-danger">{hiba}</div>}
      <div className="col"><input name="name" className="form-control" value=
{form.name} onChange={handleChange} required /></div>
      <div className="col"><input name="email" type="email" className="form-
control" value={form.email} onChange={handleChange} required /></div>
      <div className="col-auto"><button className="btn btn-
primary">Hozzáadás</button></div>
    </form>
  );
}
```

Sablonkód – közös API segéd

```
// src/api.js
const API = import.meta.env.VITE_API_URL ?? '/api';

async function request(path, options = {}) {
  const res = await fetch(API + path, {
    headers: { 'Content-Type': 'application/json', Accept: 'application/json'
},
    ...options,
  });
  if (!res.ok) throw new Error(`${options.method ?? 'GET'} ${path} →
${res.status}`);
  return res.status === 204 ? null : res.json();
}

export const apiGet = (path) => request(path);
export const apiPost = (path, data) => request(path, { method: 'POST', body:
JSON.stringify(data) });
export const apiPut = (path, data) => request(path, { method: 'PUT', body:
JSON.stringify(data) });
export const apiDelete = (path) => request(path, { method: 'DELETE' });
```

Ha a projekt axiost használ: `const { data } = await axios.get('/api/suppliers');` és `await axios.post('/api/suppliers', form);` – az axios magától JSON-t alakít, és 4xx/5xx-nél hibát dob.

5. Adatbázis elérése éles szerveren

Frontendesként az adatbázist szinte mindig az API-n keresztül éred el, nem közvetlenül. Közvetlen hozzáférést (jelszót, SSH kulcsot) a csapattól kapsz, és éles adatbázisban engedély nélkül soha nem módosítasz.

Aranyszabályok

- Éles (production) adatbázisban csak **olvasol** (`SELECT`), hacsak kifejezetten nem kérik másképp.
- Fejleszteni és próbálgatni a **helyi** vagy a **teszt (staging)** adatbázison kell.
- `UPDATE` / `DELETE` előtt: mentés (backup), és előbb ugyanazzal a `WHERE` -rel egy `SELECT` , hogy lásd, mely sorokat érinti.
- Jelszót, `.env` tartalmát soha ne tedd Gitbe, chatbe vagy képernyőképre.

1. út: az API-n keresztül (frontendesként ez a fő út)

- Nyisd meg az oldalt, F12 → **Network**: látod az adatot pontosan úgy, ahogy a backend adja.
- **Postman / Insomnia / Thunder Client** (VS Code bővítmény): bármelyik végpontot kézzel meghívhatod (GET, POST), tokennel együtt.
- **API dokumentáció**: Swagger (`/api/documentation`), Postman collection vagy a README.

2. út: webes felület (phpMyAdmin / Adminer)

Sok szerveren van webes adatbázis-kezelő, pl. `https://ceg.hu/phpmyadmin` . Belépsz a kapott felhasználóval, bal oldalt kiválasztod az adatbázist, majd a táblát: **Tallózás** fül = tartalom, **Szerkezet** fül = oszlopok, **SQL** fül = saját lekérdezés.

3. út: SSH + parancssor

```
ssh felhasznalo@szerver.ceg.hu            # belépés a szerverre
cd /var/www/projekt                        # a projekt mappája (kérdezd meg)
cat .env | grep DB_                        # adatbázis neve, felhasználója

mysql -u db_felhasznalo -p adatbazis_neve  # jelszót kér
```

```
SHOW TABLES;                       -- táblák listája
DESCRIBE suppliers;                -- a tábla oszlopai
SELECT * FROM suppliers LIMIT 10;  -- első 10 sor (LIMIT-et mindig írj!)
SELECT COUNT(*) FROM suppliers;
exit;
```

4. út: SSH tunnel grafikus programmal (a legkényelmesebb)

Az adatbázis általában nem érhető el közvetlenül az internetről, csak a szerveren belül. Az SSH tunnel „átvezeti” a saját gépedre.

1. Telepíts egy kliensprogramot: **DBeaver** (ingyenes), **HeidiSQL** (Windows), vagy **TablePlus**.
2. Új kapcsolat → MySQL → **SSH** fül: szerver címe, SSH felhasználó, jelszó vagy kulcsfájl.
3. **Main** fül: host `127.0.0.1` , port `3306` , adatbázis neve, DB felhasználó és jelszó (ezek a szerver `.env` -jében vannak).
4. Test Connection → Mentsd el. Ezután táblázatban böngészheted az adatokat.

Parancssorból ugyanez: `ssh -L 3307:127.0.0.1:3306 felhasznalo@szerver.ceg.hu` – utána a saját gépeden a `127.0.0.1:3307` címen éred el.

5. út: Laravel tinker a szerveren

```
ssh felhasznalo@szerver.ceg.hu
cd /var/www/projekt
php artisan tinker
```

```
App\Models\Supplier::count();
App\Models\Supplier::latest()->take(5)->get();
App\Models\Supplier::where('city', 'Budaörs')->get();
```

Hasznos még: `php artisan db:show` (adatbázis és táblák), `php artisan db:table suppliers` (egy tábla oszlopai), `php artisan route:list` (az összes API végpont).

6. Ha a backendhez is hozzá kell nyúlni (Laravel API)

Ha a React oldalnak kellene egy végpont, ami még nincs, négy fájlt érintesz: migráció (ha új tábla kell), model, controller, `routes/api.php`. A React oldalon utána csak az URL-t hívod.

```
php artisan make:model Supplier -mc      # model + migráció + controller
php artisan migrate                      # tábla létrehozása
php artisan route:list --path=api        # ellenőrzés
```

```
// app/Models/Supplier.php
protected $fillable = ['name', 'email', 'city'];

// routes/api.php  (Laravel 11+: előtte egyszer php artisan install:api)
use App\Http\Controllers\SupplierController;
Route::get('/suppliers', [SupplierController::class, 'index']);
Route::post('/suppliers', [SupplierController::class, 'store']);

// app/Http/Controllers/SupplierController.php
public function index()
{
    return Supplier::orderBy('name')->get();          // automatikusan JSON
}

public function store(Request $request)
{
    $data = $request->validate([
        'name'  => 'required|string|max:255',
        'email' => 'required|email|unique:suppliers,email',
```

```
        'city'  => 'nullable|string|max:100',
    ]);
    return response()->json(Supplier::create($data), 201);
}
```

A `routes/api.php` végpontjai automatikusan `/api` előtagot kapnak, így a React-ből `GET /api/suppliers` és `POST /api/suppliers`. Validációs hibánál 422-t ad vissza, a hibákkal az `errors` mezőben: Reactben ezt így írod ki: `Object.values(valasz.errors).flat().join(', ')`.

Tesztelés: előbb Postmanben vagy a böngészőben (`/api/suppliers`), és csak ha ott jó, akkor kötöd be a React oldalba.